Multi-Voltage Crypto Core: RTL to GDSII (Vol. 1)

32nm Multi-Voltage ASIC Design Reference

(0.95V / 0.75V Dual Supply Physical Design Methodology)

The Complete Implementation Handbook

Author: Ajay Mallesh

Reviewer: Gemini Pro 3.1 & Ajay Mallesh

Version: Version 2.0 (2026)

Date: 23/05/2026

“This project is generated with Gemini Pro 3.1 and crafted by Ajay Mallesh with 💔.”

Technology Focus

Parameter	Specification
1. Technology Node	32nm CMOS
2. Design Type	Multi-Voltage ASIC
3. Core Type	Crypto Accelerator
4. Instance Count	~600K Instances (Pre-Synthesis)
5. Voltage Domains	0.95V / 0.75V
6. Flow Style	RTL → GDSII
7. Primary Tools	Synopsys / Cadence
8. Timing Methodology	MCMM
9. Signoff Style	PrimeTime / StarRC
10. Power Strategy	Multi-Domain Mesh PDN
11. Clocking	CTS + NDR
12. Target Audience	Physical Design Engineers / ASIC Architects

Author: Ajay Mallesh

Email: ajaymalavalli912@gmail.com

LinkedIn: www.linkedin.com/in/ajaymallesh

Legal Disclaimer

The methodologies, TCL examples, UPF constructs, synthesis flows, timing constraints and physical implementation strategies described in this document are intended exclusively for educational and research purposes.

Tool commands and flows may vary depending on:

- tool version,
- process node,
- foundry NDA constraints,
- technology LEF/DEF structure,
- extraction deck version,
- signoff correlation methodology,
- and EDA vendor implementation changes.

The author assumes no responsibility for:

- silicon failure,
- timing closure loss,
- LVS/DRC violations,
- power collapse,
- EM degradation,
- tape-out delay,
- or fabrication loss arising from improper application of the concepts described herein.

Table of Contents

Project Introduction & Overview

Multi-Voltage Crypto Core Architecture

The crypto_top project is a high-performance ASIC cryptographic accelerator designed using 32nm CMOS technology. It contains around 30K standard cells.
Cryptographic operations such as AES-256 and SHA-3 require high-speed computation, while control logic and peripheral interfaces operate at lower speeds. To reduce power consumption and meet SoC thermal constraints, the design uses a dual-voltage architecture based on voltage scaling.
The design is divided into two voltage domains:

High-Performance Domain (0.95V / VDD)

This domain contains the main datapath and computational engines. It operates at 0.95V to achieve maximum performance and timing closure using saed32rvt_ss0p95v125c.db libraries.

Low-Power Domain (0.75V / VDDL)

This domain includes control logic and low-speed peripherals. Operating at 0.75V significantly reduces dynamic power consumption. It uses saed32rvt_ss0p75v125c.db libraries optimized for low power.

Physical Design Challenges

The multi-voltage architecture introduces several backend design complexities, including:

- UPF-based power intent integration
- Level shifters for signals crossing voltage domains
- Dedicated voltage areas and power planning
- MCMM timing analysis across multiple operating conditions

Tools

- Synopsys Design Compiler
- Synopsys ICC2

Chapter 1: Files and Directories (Linux)

1.1. Directory creations

- Command: mkdir crypto_top
- cd crypto_top/

1.2. Files creations

- Command: gvim <file_name>
- cat <file_name>
- nano <file_name>

1.3. Automated Script

- Source setup_env.tcl

** This script will create all the required directories

Chapter 2: RTL Implementation

2.1 RTL File Implementation Logic

For a multi-voltage design, the RTL logic must be cleanly partitioned into distinct hierarchical blocks based on their target voltage.

In this crypto_top design, the architecture is split into two main sections:

1. **The High-Performance Control Block:** Runs on the main **0.95V (VDD)** supply. This handles the fast I/O interfaces, main state machines and clock generation.
2. **The Low-Power Crypto Engine:** Runs on the scaled **0.75V (VDDL)** supply. This block performs the heavy, latency-tolerant encryption math.

The top-level Verilog file (**crypto_top.v**) simply acts as a wrapper that wires these two blocks together.

2.2 Level Shifters Declarations

When a signal travels from the 0.75V (VDDL) domain back to the 0.95V (VDD) domain, a **Low-to-High Level Shifter** is required. Without it, the 0.75V "High" signal won't fully turn off the 0.95V PMOS transistors, causing a short circuit (crowbar current).

How they are declared: In modern physical design, **we do not hardcode level shifters into the RTL.** The RTL just connects the wires normally.

Instead, level shifters are declared and automatically inserted using the **UPF (Unified Power Format)** file during synthesis.

Chapter 3: Synthesis

3.1 What is Synthesis?

Synthesis is the process of translating human-readable RTL code (like Verilog) into a technology-mapped gate-level netlist. The tool reads the behavioral logic and converts it into physical logic gates (AND, OR, Flip-Flops) available in the target 32nm standard cell library, while trying to meet timing and power requirements.

3.1.1 Types of Synthesis

- **Logical Synthesis:** Traditional synthesis that assumes zero wire delays. It only looks at the logic gates.
- **Physical Synthesis (Topographical):** Advanced synthesis that predicts physical layout (placement and wire lengths) to give much more accurate timing and power estimates before the actual Physical Design phase.

3.2 Synthesis used in this design

For this crypto_top core, we use **Physical (Topographical) Synthesis** using Synopsys Design Compiler (DC). Because this is a dual-rail design, the synthesis tool must also be **Multi-Voltage Aware**, meaning it reads our power intent and automatically inserts the required level shifters between the 0.95V and 0.75V domains.

3.3 Files / Inputs required for Synthesis

To run synthesis, the tool needs five critical inputs:

3.3.1 RTL

The verified Verilog files (e.g., crypto_top.v, high_speed_ctrl.v, crypto_engine_lp.v) containing the design logic.

3.3.2 .lib

The logic libraries provided by the foundry. Because we have two voltage domains, we must load libraries for both the **0.95V** standard cells and the **0.75V (VDDL)** standard cells, usually at worst-case timing corners.

3.3.3 UPF

The IEEE 1801 Unified Power Format file. This tells the tool where the VDD and VDDL domains are and rules for inserting level shifters.

3.3.4 SDC

Synopsys Design Constraints. This file defines the clock frequencies, input/output delays and false paths.

3.3.5 Scanchains

Design for Testability (DFT) setup files. These tell the tool how to stitch the flip-flops together into long "scan chains" so the chip can be tested for manufacturing defects after fabrication.

3.4 Synthesis script file - ./scripts/dc_compile.tcl

A simplified look at the main commands inside the synthesis script:

Tcl (Simplified Example) / Not to execute this alone

```
# 1. Read the design
analyze -format verilog [./inputs/crypto_top.v]
elaborate crypto_top

# 2. Load constraints and power intent
source ./inputs/crypto_top.sdc
load_upf ./inputs/crypto_top.upf

# 3. Stitch scan chains for DFT
insert_dft

# 4. Compile the design into 32nm gates
compile_ultra -scan -timing_high_effort_script
```

3.5 Generated output files

Once compile_ultra finishes, it generates the following deliverables for the Physical Design (PnR) team:

3.5.1 Golden Netlist

“.v” file containing only 32nm standard cell instances and their wire connections. All abstract RTL is gone.

3.5.2 UPF

An updated “.upf” file. The tool updates the original UPF to include the exact names and locations of the level shifters it inserted.

3.5.3 Scandef

A .def file containing the scan chain trace. This ensures the PnR tool (ICC2) knows how the flip-flops are chained together and can optimize their placement.

3.5.6 SDC (not recommended for Physical Design)

The tool generates a forward-annotated SDC file.

Note: It is generally not recommended to use this generated SDC for physical design. It often contains cluttered, auto-generated tool variables. It is best practice to pass the clean, original SDC to the PnR team.

3.6 Reports

Before handing the files off to PnR, these logs must be reviewed:

3.6.1 report_timing

Checks the critical paths. Setup timing should be met (or very close to met) here. Hold timing is generally ignored at this stage because the clock tree doesn't exist yet.

3.6.2 report_area

Checks the total cell count and silicon footprint. We verify that the design is roughly hitting our target of instances.

3.6.3 report_qor

The Quality of Results report. Provides a quick summary of the worst negative slack (WNS), total negative slack (TNS), cell count and leakage power.

3.7 Sanity checks

To sign off on synthesis, two crucial checks must pass clean:

3.7.1 check_design

Scans the netlist for logical errors. You must ensure there are **zero floating inputs** and **zero multi-driven nets** (where two outputs try to drive the same wire).

3.7.2 check_mv_design

Scans the multi-voltage architecture. This ensures that every wire crossing from the 0.75V VDDL domain to the 0.95V VDD domain successfully received a level shifter and that there are no illegal power connections.

3.8 Automated Script for synthesis

- source ./scripts/dc_compile.tcl

** This will do synthesis and generate the output files

Note: change the paths accordingly ***

Chapter 4: Floorplan

4.1 creating ndm (Refer pd_flow.tcl)

Before physical design can begin, the tool needs a unified database containing both logical timing and physical layout information. In Synopsys ICC2, this is the New Data Model (NDM). We create the NDM by merging the synthesized .v netlist, the .db timing libraries and the physical .lef/FRAM files for both the 0.95V and 0.75V domains.

4.2 Initialize Floorplan

Floorplanning establishes the physical silicon canvas for the instances.

4.2.1 Read Netlist

The tool loads the structural netlist generated by the synthesis team, mapping the logical instances to physical 32nm standard cell shapes.

4.2.2 Create Core and Die Area

We define the overall chip dimensions. The **Die Area** includes the outer I/O pad ring, while the **Core Area** is the internal region where the standard cells and macros are placed. A target utilization of roughly 75% is set to leave enough blank space for routing wires and inserting clock buffers later.

4.3 Sanity checks

4.3.1 check_floorplan

Validates that the core area is legally defined, boundaries are closed and there are no overlapping hard macros that violate placement rules.

Chapter 5: Port Placements

5.1 Block Pin Constraints - Placing ports

In a hierarchical design like the crypto_top, the external input/output signals must enter and exit the block through physical pins on the core boundary. Pin constraints define exactly which edge (top, bottom, left, right) and which metal layers the ports should use, optimizing the dataflow into the high-speed control block.

5.2 Sanity checks

5.2.1 check_pin_placement

Confirms that all pins have been successfully dropped onto the core boundary and align with the routing grid.

5.2.2 check_pre_pin_placement

A preventative check run *before* placement. It verifies that your pin constraint file is syntactically valid and that you aren't trying to place pins on an illegal metal layer or congested edge.

Chapter 6: Multi-voltage Design

6.1 UPF

We read the .upf file outputted by Synthesis into ICC2. This ensures the physical tool understands exactly which cells belong to the high-performance domain and which belong to the low-power domain.

6.2 create voltage area

We physically draw the PD_LOW_POWER region on the floorplan canvas. The tool will strictly place the 0.75V crypto engine standard cells inside this designated boundary, completely separated from the 0.95V logic.

6.3 commit UPF

This command tells ICC2 to finalize the power intent. It maps the logical UPF definitions to the physical voltage area we just drew and creates the actual power nets (VDD and VDDL) in the database.

6.4 Sanity checks

6.4.1 check_mv_design

Validates that the physical floorplan correctly supports the multi-voltage intent without illegal overlaps.

6.4.2 report_mv_cells

Generates a list of all level shifters and isolation cells, confirming they were successfully carried over from synthesis.

6.4.3 report_mv_path -level_shifter -all_path

Traces every wire crossing from the 0.75V VDDL area to the 0.95V VDD area to guarantee a level shifter physically sits between them, preventing crowbar currents.

Chapter 7: Power Delivery Network

7.1 Create Tap and bound cells

Before laying metal, we place specific physical-only standard cells:

- **Tap Cells:** Placed at regular intervals to tie the silicon substrate to ground and n-wells to power, preventing latch-up.
- **Boundary Cells (Endcaps):** Placed at the end of every standard cell row to protect the delicate logic gates from manufacturing damage.

7.2 Sanity checks

7.2.1 check_boundary_cells

Ensures all rows are properly capped and that the maximum allowed distance between tap cells is not violated.

7.3 creating PG nets

We construct the power grids (VDD, VDDL and VSS) using upper metal layers to distribute current evenly and prevent IR-drop.

For the low-power crypto region, the implementation prioritizes direct, explicit routing over complex automation. The **VDDL** mesh is generated ensuring it does not break before the inner guard band; the VDDL shapes must connect continuously edge-to-edge with the guard band to guarantee uncompromised power delivery. Furthermore, standard cell macro ring strategies are explicitly commented out in the reference scripts to strictly maintain this defined topology.

7.4 Sanity checks

7.4.1 check_pg_connectivity -check_std_cell_pins none

Verifies the macro-level power grid is fully connected to the supply ports without getting bogged down by checking millions of individual standard cell pins.

7.4.2 check_pg_missing_vias

Ensures that where vertical and horizontal power straps cross, a physical via successfully fuses them together.

7.4.3 check_pg_drc -ignore_std_cells

Checks the thick power metal routes for spacing and width Design Rule Check (DRC) violations, ensuring the foundry can manufacture the grid without shorts.

7.5 Automated script for Floorplan to PowerPlan

Rather than typing these steps interactively, the entire sequence from **Chapter 4** through **Chapter 7** is executed deterministically via a master control script:

- **source ./scripts/pd_flow.tcl**

This script handles the NDM creation, floorplan initialization, port placement, UPF commitment and the strict VDDL power grid generation in a single automated pass.

Chapter 8: Congestion checks

Before committing to the heavy, time-consuming optimization phases, we must perform a "dry run" to ensure our floorplan, macro placements and voltage areas won't cause a massive traffic jam of routing wires later on.

8.1 create_placement -floorplan

Instead of placing cells meticulously for timing, this command performs a fast, coarse placement of the ~30k standard cells. The -floorplan flag tells the tool to ignore heavy timing optimizations and simply drop the standard cells into the core (respecting the 0.95V and 0.75V voltage areas) to see how crowded the silicon canvas gets.

8.2 congestion reports

Once the cells are temporarily placed, we need to analyze if there is enough physical space (metal tracks) for all the logical connections to be wired together.

8.2.1 report_congestion -rerun_global_router

This command is critical. It triggers the global routing engine to do a quick, rough estimate of how the wires will be drawn.

- The -rerun_global_router switch forces the tool to calculate fresh routing data based exactly on the fast placement we just did.
- We review the log to identify "hotspots." If the tool reports high overflow (meaning more wires need to pass through a channel than there are physical metal tracks available), the floorplan is unroutable. You must stop, adjust your macros or pin constraints and try again.

8.3 reset_placement

Since create_placement -floorplan was only a temporary test to check for congestion hotspots, we do not want to keep it. Once we confirm the floorplan is highly routable, we execute reset_placement. This rips up the quick standard cell placement, leaving behind our clean, verified floorplan and power grid, ready for the actual, formal, timing-driven placement phase.

Chapter 9: Placement

Placement is the phase where the tool formally assigns a physical X,Y coordinate to all ~30k standard cells. The goal is to optimize logic for setup timing, minimize total wire length and strictly obey the 0.95V and 0.75V voltage boundaries, all without creating routability issues.

9.1 Source ./scripts/placement.tcl

Instead of running commands manually, the placement engine (typically place_opt) is executed via this master script. It performs global placement, legalizes the cells onto the standard cell rows and performs initial High-Effort logic optimization to fix setup timing violations.

9.2 Source mcmm_setup file - modes & scenarios

- source ./inputs/constraints/mcmm_setup.tcl

Note: make sure to add this path properly in mcmm_setup.tcl

- source ./inputs/constraints/mcmm_corners_modes.tcl

*** Make sure all paths of **mcmm & tech** in “mcmm_corners_modes.tcl” are valid ***

Before placement optimization begins, Multi-Corner Multi-Mode (MCMM) constraints must be loaded. The placement engine must "see" all operational scenarios simultaneously. For this design, it must balance the timing paths across different Process, Voltage and Temperature (PVT) corners (e.g., worst-case timing at 0.85V for VDD and 0.65V for VDDL at 125°C, versus best-case at -40°C). Fixing a path in one corner without MCMM loaded can easily break it in another.

9.3 Sanity checks

9.3.1 check_legality

Validates that every single standard cell is placed legally. This means:

- Cells are aligned to the manufacturing site rows.
- Zero cells are overlapping.
- Zero 0.75V logic cells have accidentally leaked out of the PD_LOW_POWER voltage area.

9.4 Reports

After placement, timing violations are expected, but physical Design Rule Checks (DRC) like transition and capacitance must be heavily scrutinized.

9.4.1 report_constraints -all_violators -max_transition -significant_digits 6

Identifies nets where the signal takes too long to toggle from 0 to 1 or 1 to 0 (slew rate). Slow transitions cause sluggish performance and massive short-circuit power dissipation.

9.4.2 report_constraints -all_violators -max_capacitance -significant_digits 6

Identifies cells trying to drive a load capacitance that exceeds the maximum limit characterized in the .lib files. This happens when a net is too long or a cell has too many fanouts.

9.5 Fixes for max trans

If the tool's auto-optimization leaves max transition violations, they must be resolved. Transition degradation is mostly caused by excessive RC wire delay.

9.5.1 Upsize cells (Not Recommended) - script: ./script/upsize_driver.tcl

Swaps the driving cell for a larger footprint cell with stronger drive strength. **Not recommended manually** because blindly upsizing steals area, increases power consumption and often creates congestion hotspots in dense areas.

9.5.2 Insert Buffers (Not Recommended) - script: ./script/insert_buffer.tcl

Cuts a long, resistive wire in half by inserting a buffer to restore the signal slew. **Not recommended manually** at this stage because the clock tree doesn't exist yet; manual buffers can severely complicate CTS (Clock Tree Synthesis) later.

9.5.3 Magnet placement

A physical constraint technique. You define a "magnet" (often a macro or a critical level shifter) and the tool pulls all logically connected standard cells physically closer to it. Shorter wires mean less resistance, naturally fixing the transition violation.

9.5.4 Create_bounds

Forces a specific group of critical cells into a tightly restricted physical box. By caging the cells together, you prevent the placement engine from spreading them out across the die during optimization, controlling wire length.

9.5.5 Refine_placement

An incremental command run after applying bounds or magnets. It gently shifts surrounding cells to legalize the new constraints without destroying the overall placement optimization you already achieved.

9.5.6 Path grouping

Assigns a higher "weight" or priority to specific failing paths (for example, the data bus crossing from VDDL to VDD). This forces the optimization engine to spend more computational effort fixing transitions on those specific nets at the expense of non-critical paths.

9.6 Fixes for max cap

Max capacitance violations are primarily driven by fanout (one cell trying to drive too many receivers).

9.6.1 Upsize cells (Not Recommended) - script : ./script/upsize_driver.tcl

Using a stronger gate to handle the large capacitive load. As before, this is a brute-force approach that degrades area and power metrics.

9.6.2 Insert Buffers (Not Recommended) - ./script/insert_buffer.tcl

Isolating the heavy load from the original driver by buffering the net.

9.6.3 Split load

Instead of one cell driving 40 endpoints, you insert parallel buffers. Buffer A drives 20 endpoints and Buffer B drives the other 20. This halves the capacitance seen by the drivers, fixing the DRC violation efficiently.

9.6.4 Cloning

A highly effective logic transformation. You instruct the tool to create an exact duplicate (clone) of the struggling logic cell. The tool then routes the inputs to both cells and distributes half the output fanout to the original cell and half to the clone. This perfectly resolves max cap issues on critical control signals without adding unnecessary buffer delays.

Chapter 10: Clock Tree Synthesis

Up until this phase, the clock was "ideal" (zero delay, infinite drive strength). Clock Tree Synthesis (CTS) physically builds the clock distribution network, inserting buffers and routing wires to deliver the clock signal to all flip-flops simultaneously. In a multi-voltage design, clock domains crossing from 0.95V to 0.75V must be heavily scrutinized to minimize insertion delay differences caused by the voltage drop.

10.1 Type of CTS used - Fish Bone

For this high-speed crypto core, we utilize a **Fishbone (or Spine) Clock Routing** topology. Instead of a highly branched, unstructured tree, the main clock trunk runs down the center of the core (the spine), with horizontal branches extending outward (the ribs) to drive the local standard cells. This structural approach minimizes On-Chip Variation (OCV) and provides extremely low, predictable skew across the high-performance logic.

10.2 source ./scripts/cts_script.tcl

This master script executes the complete clock building process for the standard cells. The execution flow is strictly controlled through the following stages:

- **NDR Setup:** Applies double-width and double-spacing rules (iccrm_clock_double_spacing) to the clock nets, restricting them to routing layers M4 and M5 to shield against crosstalk and reduce RC delay.
- **Clock Constraints & MCMM:** Max transition is clamped at 0.15. Target skews are corner-specific (e.g., a relaxed 0.05ns for slow-slow corners at 125°C/-40°C and a tighter 0.02ns for fast-fast corners). Setup/hold uncertainties are also initialized.
- **CRPR Validation:** Clock Reconvergence Pessimism Removal (CRPR) is enabled to ensure the timing engine doesn't artificially penalize common clock path segments.
- **Level Shifter Insertion:** Crucially, exactly **2 level shifters** are manually inserted on the clock nets crossing from the 0.95V spine into the 0.75V VDDL low-power engine. This is done explicitly before building the tree for two vital reasons:
    1. **Electrical Integrity:** A clock signal crossing between 0.95V and 0.75V requires a level shifter to safely step the voltage. Without it, the receiving 0.75V logic gates won't switch correctly, leading to direct short-circuits (crowbar currents) from power to ground.
    2. **Preventing Skew Disasters:** Level shifters are physically complex cells that introduce a significant amount of delay. If the tool automatically inserts them *after* the tree is already built, that sudden chunk of added

delay completely destroys the clock skew. By explicitly inserting them *first*, the CTS engine "sees" their inherent delay from the very beginning and builds the rest of the 0.95V tree with matching delay buffers to perfectly balance the overall timing.

- **CTS Execution Stages:** The tree is built incrementally using **clock_opt -to build_clock**, routed with **clock_opt -to route_clock and** finally optimized for high-effort hold timing and scan reordering via **clock_opt -final_opto**. The database is saved at each milestone.

10.3 Issues / violations

Common violations post-CTS include:

- **Max Skew:** The difference in clock arrival time between the earliest and latest flip-flop is too large.
- **Max Insertion Delay:** The clock takes too long to travel from the port to the flip-flops, making it highly vulnerable to PVT variations.
- **Pulse Width / Duty Cycle Distortion:** The high/low shape of the clock wave degrades as it travels through unequal rise/fall buffer paths.

10.4 Fixes for Violations

To fix clock violations without ruining the tree, we apply specific CTS constraints:

- **NDR Usage:** Enforcing strict double-spacing rules on clock nets to shield them from neighbouring data lines.
- **Restricted Cell Lists:** Forcing the tool to use only specific, highly symmetrical clock buffers and inverters rather than standard logic buffers.
- **Manual Guide Buffers:** Manually dropping a strong clock buffer at a divergence point to force the tool to balance a stubborn branch.

10.5 Fixes for Hold / setup

Once the clock is "real," it takes time to reach the flip-flops, causing massive shifts in timing.

- **Setup Fixes:** Setup paths fail if data arrives too late. The tool fixes this by downsizing data path logic, cloning heavily loaded cells, or utilizing **Useful Skew** (deliberately delaying the clock to the receiving flip-flop to give the data more time to arrive).
- **Hold Fixes:** Hold paths fail if data arrives too fast (racing past the clock). The tool exclusively fixes this by **inserting delay buffers** into the data path to slow the signal down.

10.6 Sanity checks

Verify the clock tree structure by ensuring there are no unrouted clock nets, no overlapping clock buffers and that the clock signal cleanly passes through the newly inserted level shifters when crossing into the 0.75V domain.

10.7 Reports

report_clock_tree → analyze the longest and shortest paths and

report_clock_timing -type skew → to ensure global and local skew targets are met before routing the data nets.

Chapter 11: Routing

With the standard cells placed and the clock tree built, the remaining task is to wire all the logic together. The routing engine must physically connect millions of pins using the available metal layers while avoiding congestion, shorts and physical manufacturing rules.

11.1 source ./scripts/route.tcl

Detailed routing is executed via this master script (typically running the route_opt or route_auto commands). It operates in three distinct phases:

1. **Global Routing:** Divides the chip into grids and plans the general path for every wire, managing overall congestion.
2. **Track Assignment:** Assigns those general paths to specific metal tracks (e.g., Metal 3, Metal 4) in the routing channels.
3. **Detailed Routing:** Connects the actual standard cell pins, navigating around obstacles, vias and adjacent wires to resolve all physical DRCs.

11.2 Sanity checks

11.2.1 check_design -checks pre_route_stage

Before the heavy detailed routing begins, this crucial check ensures the database is clean. It verifies that placement legality holds, the multi-voltage power grid (VDD and VDDL) is fully intact and the clock tree is fully routed and fixed. Launching the router on a broken database wastes days of runtime.

11.3 Reports

11.3.1 check_routability

This command analyzes the routing grids for DRC (Design Rule Check) hotspots. It ensures that the router did not leave **open nets** (unrouted wires) or **short circuits** (wires touching) that cannot be resolved in subsequent optimization passes.

11.4 Antenna Rule

During plasma etching in manufacturing, long metal wires act like antennas, accumulating static electrical charge. If this charge has nowhere to go, it blows out the delicate thin-oxide gate of the transistor it connects to.

The router detects and fixes Antenna Violations using two automated methods:

1. **Layer Hopping:** Breaking the long wire and routing it up to the highest metal layer (which is etched last) to interrupt charge accumulation.
2. **Antenna Diodes:** Inserting a reverse-biased diode near the transistor gate to bleed the accumulated charge safely into the silicon substrate.

Chapter 12: Signoff

Signoff is the final phase of the ASIC design cycle. The physical database is locked and rigorous checks are performed using specialized signoff tools to guarantee the chip will function correctly after manufacturing.

12.1 source ./scripts/signoff_script.tcl

The signoff process involves exporting the physical design data for final verification across several domains:

- **Parasitic Extraction (StarRC):** Extracts the final, highly accurate resistance and capacitance values (RC) of every routed wire.
- **Static Timing Analysis (PrimeTime):** Uses the extracted RC data to run a final timing check across all MCMM corners. The design must have **zero setup and zero hold violations** across both the 0.95V and 0.75V domains.
- **Physical Verification (IC Validator / Calibre):**
    - **DRC (Design Rule Check):** Ensures spacing, width and via rules strictly meet the 32nm foundry manual.
    - **LVS (Layout Versus Schematic):** Extracts the physical shapes back into a netlist and compares it against the golden synthesis netlist to ensure zero missing or shorted connections.
- **IR Drop / EM Signoff (RedHawk / Voltus):** Confirms the power mesh—especially the 0.75V VDDL grid connected to the guard band—delivers adequate voltage to all instances under peak switching activity without electromigration failure.

Once all signoff checks report zero violations, the design achieves **Tapeout Readiness** and the final GDSII file is generated for mask creation at the foundry.

Chapter 13: Advance IR drop

13. Power Integrity & Dynamic IR-Drop Signoff

Even if Static Timing Analysis reports zero violations, the silicon can still fail if the power delivery network (PDN) collapses under heavy computational load. Because modern ICC2 environments rely on the Ansys RedHawk-SC fusion integration for native analyze_rail commands, missing this integration requires Power Integrity (PI) analysis to be performed as an external signoff step using dedicated standalone tools (e.g., PrimePower, Voltus, or standalone RedHawk).

13.1 Introduction & Physics/Theory

When thousands of logic gates in the crypto engine toggle simultaneously, they pull a massive surge of current (di/dt) through the resistive metal tracks of the PDN. According to Ohm's Law ($V_{drop} = I * R$), this current draw causes the local supply voltage to sag. If the 0.75V VDD_LP supply temporarily drops to 0.68V, the standard cells switch slower, potentially violating setup time limits that were verified under ideal 0.75V conditions.

13.2 Exporting the Design for External Analysis

To run power analysis outside of ICC2, the physical and parasitic data must be accurately exported. The external PI tool requires a snapshot of the fully routed design, the parasitic resistances of the power grid and the switching activity.

Tcl

```
# Export the physical layout
write_def -version 5.8 -routing -all_blocks reports/crypto_top.def
# Export the parasitic RC data
write_parasitics -format SPEF -output reports/crypto_top.spef
# Optional: Generate a power report in ICC2 for baseline estimation
report_power -scenarios [current_scenario] > reports/icc2_baseline_power.rpt
```

13.3 Files / Inputs Required for Signoff

The external Power Integrity team will ingest the following files:

- **DEF (.def):** Contains the physical placement of all standard cells, macros and the exact metal routing of the power grid.
- **SPEF (.spef):** Contains the exact resistance and capacitance values of the metal routes.
- **.lib (Liberty):** Contains the internal power models and current draw characteristics for the 0.95V and 0.75V standard cells.

- **VCD/FSDB:** A Value Change Dump generated by the verification team running a gate-level simulation of an active encryption payload. This provides the exact switching timestamp of every net for dynamic analysis.

13.4 Validating the VDDL Topology

This standalone analysis proves the physical viability of your custom power plan. The PI tool specifically evaluates the VDD_LP low-power region to ensure the power mesh connecting directly to the inner guard band is delivering uncompromised current. If the manual topology (which intentionally bypassed macro ring automation) is too resistive, the tool will flag massive voltage droop hotspots in the center of the crypto engine.

13.5 Heatmaps & Signoff Reports

The external tool will generate two primary deliverables:

- **Dynamic Voltage Drop Heatmap:** A visual color map of the die highlighting areas where the voltage drops below the accepted threshold (e.g., a drop greater than 5%).
- **Electromigration (EM) Reports:** Identifies metal tracks or vias where the current density (**J**) exceeds foundry limits, which could cause the wire to degrade or melt over time.

13.6 Fixes for IR-Drop Violations

If the PI team flags unacceptable voltage drops, fixes must be implemented back in the ICC2 environment via Engineering Change Orders (ECOs):

- **Adding Decap Cells:** Decoupling capacitors act like tiny local batteries. They are inserted into empty spaces near hotspots to supply instantaneous charge during switching bursts.
- **Thickening Local Straps:** Manually dropping additional power vias or widening specific M7/M8 straps over the starving region to lower the resistance (***R***).
- **Spreading the Logic:** Applying density padding constraints in ICC2 and incrementally refining placement to break up dense clusters of high-drive standard cells.

Chapter 14: Hard Macro Conversion (Block Abstraction)

Once the crypto_top core has achieved timing closure, clean IR-drop and zero DRC/LVS violations, it can be "hardened." Hardening means sealing the internal 600,000 standard cells and converting the entire block into a single, black-box component (a Hard Macro) to be placed inside a larger System-on-Chip (SoC).

The top-level SoC tools do not need to see your internal gates; they only need to know the physical boundaries, the pin locations, the power rails and the timing of the input/output ports.

14.1 Physical Abstraction (Creating the NDM / FRAM View)

To use the macro in a top-level ICC2 flow, we must generate a lightweight physical abstraction. This view hides the internal standard cells and only exposes the core boundary, port geometries and routing blockages (to prevent the top-level router from accidentally dropping vias into your internal logic).

Tcl

```
# Open the fully routed and signed-off block
open_block crypto_top_routed

# Create the abstract view (Frame) for top-level hierarchical design
create_abstract -read_only
```

This command generates an abstract NDM view. It identifies which metal layers are saturated by the crypto core and creates placement/routing blockages, leaving only the highest empty metal layers available for top-level "over-the-cell" routing.

14.2 Timing Abstraction (Extracted Timing Model - ETM)

Loading 30k internal gates into the top-level Static Timing Analysis (STA) would crash the tool or take days to run. Instead, we generate an Extracted Timing Model (ETM), which reduces the entire block's timing to just the setup/hold requirements of the input pins and the clock-to-q delays of the output pins.

This is typically done in PrimeTime:

Tcl

```
# Inside PrimeTime
read_verilog crypto_top_routed.v
read_parasitics crypto_top.spef
# ... load constraints and link ...

# Extract the timing model to generate a liberty file
extract_model -format {lib db} -output crypto_top_macro
```

This creates crypto_top_macro.lib and .db. The SoC synthesis and timing teams will use these exactly like they use standard cell libraries.

14.3 Multi-Voltage & Power Pin Promotion

Because this is a dual-voltage macro, the top-level SoC must provide both 0.95V and 0.75V to your block.

During the abstraction process, your internal power meshes (specifically the VDD_DEFAULT and VDD_LP shapes on the uppermost metal layers) must be promoted as logical **Supply Ports**.

- The top-level floorplanner will look at your .lef or NDM abstract to find the exact X,Y coordinates of your VDDL guard band mesh.
- The top-level Power Delivery Network (PDN) will then drop massive power vias directly onto those exposed geometries to feed your crypto engine.

14.4 Exporting Industry-Standard Handoff Files

If the SoC is being assembled using third-party tools (not strictly ICC2), you must export standard interchange formats rather than just an NDM. A complete "Hard Macro IP Kit" consists of four files:

Tcl

```
# 1. GDSII: The complete, flattened physical layout for manufacturing
write_gds -design crypto_top_routed ./export/crypto_top.gds

# 2. LEF (Library Exchange Format): The physical boundary and pin locations
write_lef ./export/crypto_top.lef

# 3. Verilog (Black-box): A structural shell for top-level LVS
# (Contains only the module declaration and port list, no internal logic)
write_verilog -compress false -hierarchy abstract ./export/crypto_top_bb.v
```

The Final Hard Macro Deliverables:

1. **crypto_top.gds** (Manufacturing shapes)
2. **crypto_top.lef** (Physical pins and blockages for the top-level router)
3. **crypto_top.lib / .db** (Timing models for the top-level STA)
4. **crypto_top_bb.v** (Empty wrapper for LVS)
5. **crypto_top.upf** (The power intent, declaring the dual VDD/VDDL pins)

Appendix A: 100 General Physical Design Interview Scenarios

This section covers 50 common, real-world problems that happen during Physical Design. These are organized by the stages of the ASIC flow.

Section 1: Floorplanning & Power Planning

1. Scenario: You place a large memory macro right in the dead center of the core area. What happens to the design?

- **Answer:** It acts like a giant wall. Data trying to travel from the left side of the chip to the right side must take a long detour around the macro. This creates very long wires, bad timing and routing traffic jams.

2. Scenario: You put a macro touching the absolute edge of the core boundary. What is the issue?

- **Answer:** The router cannot access the pins on the edge side of the macro. You must leave a small gap between the macro and the core boundary for wires to travel through.

3. Scenario: You forgot to add Tap cells to your floorplan. What will happen when the chip turns on?

- **Answer:** Tap cells connect the base of the silicon to power and ground. Without them, a sudden voltage spike will create a short-circuit inside the silicon (called Latch-up), which permanently burns the chip.

4. Scenario: You decided to use huge, thick metal straps for the entire power grid to make sure there is no voltage drop. What is the negative side effect?

- **Answer:** Power straps take up metal routing tracks. If you use too much metal for power, there won't be enough empty space left for the router to connect the actual signal wires, leading to a blocked design.

5. Scenario: You did not put enough power straps. What happens?

- **Answer:** The resistance of the power grid goes up. When many cells turn on at once, the voltage drops (IR Drop). If the cells don't get full voltage, they switch slowly and timing fails.

6. Scenario: Why do we put Endcap (Boundary) cells at the end of every standard cell row?

- **Answer:** During manufacturing, the edges of the cell rows can get damaged or stressed. Endcap cells are "dummy" cells with no logic inside. They sit on the edges to take the damage and protect the real logic gates inside the row.

7. Scenario: You placed all your high-speed, power-hungry macros in the top-right corner of the chip. Why is this bad?

- **Answer:** This creates a "thermal hotspot." All the heat is trapped in one small area, which can slow down the transistors or physically damage the chip. Macros should be spread out evenly if possible.

8. Scenario: What happens if you do not add a routing blockage over a hard macro?

- **Answer:** The tool might try to route standard signal wires straight over the macro and drop vias into it. This will short-circuit the internal wiring of the macro.

9. Scenario: The input ports (pins) are placed on the top of the chip, but the logic they connect to is placed at the bottom. What is the result?

- **Answer:** The tool has to draw a massive wire across the whole chip. This creates huge wire delay and setup timing violations right from the start.

10. Scenario: You forgot to connect VSS (Ground) to a macro block. What happens?

- **Answer:** The macro will not turn on. Electricity needs a complete circle (from Power to Ground) to flow.

Section 2: Placement & Congestion

11. Scenario: The tool placed 50,000 cells in one tiny corner of the chip. What is this called and why is it bad?

- **Answer:** This is called local congestion. The router will fail because there are not enough metal tracks in that tiny area to connect all those cells.

12. Scenario: How do you fix a heavily congested spot in placement?

- **Answer:** You use "Placement Blockages" or "Cell Padding" (adding invisible empty space around cells). This forces the tool to spread the cells out into wider areas.

13. Scenario: A wire is too long, causing the signal to travel very slowly. The tool reports a "Max Transition" violation. How do you fix it?

- **Answer:** You can cut the long wire in half by inserting a buffer in the middle. You can also upsize the driving cell so it pushes the signal harder.

14. Scenario: One small cell is trying to drive 100 other cells. The tool reports a "Max Capacitance" violation. How do you fix it?

- **Answer:** You can "clone" the driver. You make an exact copy of the driving cell and let the first cell drive 50 and the cloned cell drive the other 50.

15. Scenario: Two cells that talk directly to each other are placed very far apart by the tool. How do you force them together?

- **Answer:** You can use "Bounds" to lock them into the same small box, or define them as a "Magnet" so they pull towards each other.

16. Scenario: Your standard cell utilization is 90% right after placement. Is this good?

- **Answer:** No, it is too high. The tool needs empty space to add buffers during clock building and routing. If you start at 90%, the chip will be completely full and fail later. Target 70-75% to start.

17. Scenario: You fix all your timing violations by replacing small cells with huge, high-drive cells. What is the negative side effect?

- **Answer:** High-drive cells take up more physical space (causing congestion) and they leak much more power.

18. Scenario: Why is it bad if the tool places flip-flops perfectly in a straight line?

- **Answer:** If they are in a straight line, they will all draw power from the exact same local power strap at the exact same time when the clock hits. This creates a massive localized voltage drop.

19. Scenario: You have a Max Transition violation, but you cannot insert a buffer because there is no physical space left. What else can you do?

- **Answer:** Change the driving cell to a Low-VT (LVT) cell. LVT cells switch much faster without needing to be physically bigger.

20. Scenario: What is Scan Reordering and why do we do it during placement?

- **Answer:** Scan chains connect all flip-flops for testing. If connected in random order, the wires will cross back and forth across the chip, causing huge congestion. Reordering connects them based on their physical placement, keeping the wires short.

Section 3: Clock Tree Synthesis (CTS)

21. Scenario: The clock takes 5ns to travel from the clock port to the flip-flops. This is called High Insertion Delay. Why is it bad?

- **Answer:** The longer the clock travels through buffers, the more sensitive it is to temperature and voltage changes on the chip. This makes timing very unpredictable.

22. Scenario: The clock reaches Flop A at 1ns, but reaches Flop B at 2ns. What is this called and what does it cause?

- **Answer:** This is called Clock Skew. It is very dangerous because it can cause Hold timing violations (data races past the clock).

23. Scenario: Why do we use "Double Spacing" rules when routing clock nets?

- **Answer:** Clocks toggle constantly. If a data wire is routed too close, the clock will create noise (crosstalk) on the data wire. Double spacing keeps them safely apart.

24. Scenario: Why do we use special "Clock Buffers" instead of normal "Logic Buffers" to build the tree?

- **Answer:** Normal buffers might have a fast rise time but a slow fall time. Clock buffers are built to be perfectly symmetrical, so the clock wave stays perfectly square as it travels.

25. Scenario: The router put the main clock wires on Metal 1 and Metal 2. Why is this a mistake?

- **Answer:** Lower metals are very thin and have high resistance. The clock is the most important signal; it must be routed on thick, upper metals (like M5 or M6) to travel fast.

26. Scenario: What does an ICG (Integrated Clock Gating) cell do?

- **Answer:** It acts like an on/off switch for the clock. If a block of logic is not being used, the ICG shuts off the clock to that block to save power.

27. Scenario: An ICG cell is placed very far away from the flip-flops it controls. What is the risk?

- **Answer:** The "enable" signal telling the clock to turn on might arrive too late, missing the clock edge entirely.

28. Scenario: Why do we wait until *after* CTS to fix Hold violations?

- **Answer:** Hold timing depends entirely on when the clock arrives. Before CTS, the clock is "ideal" (fake zero delay). If you add buffers to fix Hold early, you will be fixing fake numbers and wasting space.

29. Scenario: What is "Useful Skew"?

- **Answer:** It is a trick to fix Setup timing. If data is arriving late to a flip-flop, you intentionally delay the clock going to that flop. This gives the data extra time to get there.

30. Scenario: You used Useful Skew to delay the clock to Flop B. But Flop B sends data to Flop C. What is the new problem?

- **Answer:** Because Flop B's clock is now late, it launches its data late. This might create a new Setup violation at Flop C.

Section 4: Routing & Design Rule Checks (DRC)

31. Scenario: What is an Antenna Violation?

- **Answer:** During manufacturing, metal is etched using plasma. Long metal wires act like antennas and collect static electricity. If the wire connects to a sensitive transistor gate, the spark will burn the gate and destroy it.

32. Scenario: How does the router fix an Antenna Violation?

- **Answer:** It uses "Layer Hopping" (breaking the wire and jumping to a higher metal layer to stop the charge from building up) or it inserts an "Antenna Diode" to safely drain the electricity into the ground.

33. Scenario: Two wires are routed perfectly parallel and very close together for a long distance. What happens?

- **Answer:** They act like a capacitor. If one wire switches, it can slow down or speed up the signal on the other wire. This is called Crosstalk Delay.

34. Scenario: How do you fix Crosstalk Delay?

- **Answer:** Move the wires further apart (increase spacing), upsize the driver to overpower the noise, or put a grounded metal wire between them to act as a shield.

35. Scenario: The router leaves an "Open Net" (a wire that didn't connect). Why does this happen?

- **Answer:** The destination pin might be blocked by other wires, or the routing channel is 100% full and there are no empty tracks left to use.

36. Scenario: Why do we force horizontal routing on Metal 3 and vertical routing on Metal 4?

- **Answer:** This is called preferred routing direction. It creates a grid. If we allowed wires to go any direction on any layer, they would block each other constantly and cause massive congestion.

37. Scenario: A single via is connecting a very important, high-current wire. What is the risk?

- **Answer:** Vias have high resistance. A single via might fail or melt. High-current wires should use "Double Vias" (two vias side-by-side) for safety.

38. Scenario: What is Electromigration (EM)?

- **Answer:** When too much current flows through a thin wire for a long time, the electricity literally pushes the metal atoms out of place. Eventually, the wire breaks or shorts out.

39. Scenario: How do you fix an EM violation?

- **Answer:** Make the metal wire wider, or route the signal using multiple metal layers at the same time to spread the current out.

40. Scenario: The tool reports a "Short" violation. What does this mean?

- **Answer:** Two completely different signals were routed so close together that their metal shapes are physically touching. This ruins both signals.

Section 5: Timing Closure

41. Scenario: A path has a Setup violation. What is the fastest way to fix it without touching the wire?

- **Answer:** Swap the cells on the path from High-VT (HVT) or Regular-VT (RVT) to Low-VT (LVT). LVT cells process data much faster.

42. Scenario: If LVT cells are so fast, why don't we build the whole chip out of them?

- **Answer:** LVT cells have massive "leakage power." They waste a lot of electricity even when the chip is just sitting idle doing nothing.

43. Scenario: A path has a Hold violation (data is moving too fast and racing past the clock). How do you fix it?

- **Answer:** You insert delay buffers into the data path. This acts like a speed bump, slowing the data down so it arrives safely after the clock edge.

44. Scenario: Why does adding buffers fix Hold, but not break the clock?

- **Answer:** Because you are adding the buffers to the *data* wire, not the clock wire.

45. Scenario: A single path has both a Setup violation AND a Hold violation at the same time. Which one do you fix first?

- **Answer:** Always fix Setup first. You fix Setup by making the path faster. Then, you fix Hold by adding a delay buffer at the very end of the path where it won't hurt the Setup time you just fixed.

46. Scenario: What is a False Path?

- **Answer:** It is a physical wire connection that exists, but data never actually travels through it during normal operation (like a reset signal that only happens once). We tell the tool to ignore it so it doesn't waste time trying to optimize it.

47. Scenario: What is a Multi-Cycle Path?

- **Answer:** Some calculations are too big to finish in one clock cycle. We tell the tool "Multi-Cycle Path 2", which means the data is allowed to take two full clock pulses to reach the end.

48. Scenario: Setup timing is failing mostly at the "Slow" temperature corner (like 125°C). Why does heat cause Setup failures?

- **Answer:** Heat increases the resistance in the silicon and metal. High resistance means signals travel slower, causing them to arrive late (Setup failure).

49. Scenario: Hold timing is failing mostly at the "Fast" temperature corner (like -40°C). Why does cold cause Hold failures?

- **Answer:** Cold temperatures make transistors switch incredibly fast. The data finishes processing so quickly that it crashes into the next flip-flop before the clock is ready (Hold failure).

50. Scenario: Your Setup margin is exactly 0.00ns (perfectly met). You have a Hold violation. If you add a delay buffer to fix Hold, it will slow the path down and cause a Setup violation. What is a smart fix?

- **Answer:** Instead of adding a new buffer, look at the driving cell. If you downsize the driving cell slightly, it will drive the signal a tiny bit slower. This might fix the Hold violation without adding enough delay to break the Setup timing.

Section 6: Advanced Timing & On-Chip Variation (STA)

51. Scenario: A path has a massive Setup violation. You check the layout and the physical wire is extremely short. How is a Setup violation even possible on a short wire?

- **Answer:** Setup depends on total delay, not just wire length. If the wire is short, the delay is coming from the cells. The driving cell likely has a massive fanout (trying to drive 50 other gates) or is heavily under-sized.

52. Scenario: The tool reports a Hold violation on a path. You look at the clock tree and realize the launch flip-flop and capture flip-flop share the exact same clock wire for 90% of their journey. Why is the tool penalizing you for clock skew?

- **Answer:** You forgot to enable **CRPR** (Clock Reconvergence Pessimism Removal). Because of On-Chip Variation (OCV), the timing engine assumes the worst case: it artificially slows down the launch clock and speeds up the capture clock. CRPR tells the tool, "Hey, this is the exact same physical wire, it can't be fast and slow at the same time," and removes the fake penalty.

53. Scenario: Why does temperature inversion happen at advanced nodes (like 16nm and below)? Why does the chip sometimes get *slower* at -40°C?

- **Answer:** In older nodes, cold temperatures always meant less resistance and faster transistors. However, at modern nodes, the threshold voltage (V_{th}) of the transistor is highly sensitive. At cold temperatures, V_{th} increases. If you are running the chip at a very low supply voltage (like 0.75V), that higher V_{th} chokes the transistor, drastically reducing the drive current (I_{ds}) and slowing the cell down.

54. Scenario: You have a Setup margin of exactly 0.00ns. You discover a Hold violation of 2ns on the exact same path. Can you fix this by just adding delay buffers?

- **Answer:** No. If you add 2ns of delay buffers to fix Hold, you will instantly create a 2ns Setup violation, bouncing back and forth forever. This means your clock skew is fundamentally broken. You must fix this architecturally by pulling the clock back on the capture flop (Useful Skew) or fixing the clock tree design.

55. Scenario: You have a data signal crossing from a 500MHz clock domain to a completely asynchronous 100MHz clock domain. PrimeTime reports a Setup violation. How do you fix the timing?

- **Answer:** You don't. This is a trick question. Asynchronous domains have clocks that drift constantly relative to each other; they will *always* violate setup/hold eventually. You must declare this a **False Path** in STA and use a structural synchronizer (like a 2-stage flip-flop synchronizer or a FIFO) in the RTL to handle the data safely.

56. Scenario: What is "Time Borrowing" and which component allows it?

- **Answer:** Flip-flops have a hard edge; data must arrive before the clock hits. **Latches** are transparent for half the clock cycle. If data arrives a little late to a latch, it doesn't fail; it just flows through and "borrows" time from the next logic stage. This is highly useful for smoothing out uneven logic paths.

57. Scenario: You see a Timing Path where the Required Time is earlier than the Launch Time. What does this mean?

- **Answer:** This is a **Half-Cycle Path**. The launch flop is triggered on the rising edge of the clock, but the capture flop is triggered on the falling edge. The data only has half a clock cycle to arrive, cutting your timing margin in half.

58. Scenario: Why do we derate the launch path "late" (slower) for Setup analysis, but "early" (faster) for Hold analysis?

- **Answer:** STA must always prove the absolute worst-case scenario. For Setup, the worst case is if the data launches extremely *late* and struggles to reach the end. For Hold, the worst case is if the data launches extremely *early* and races past the clock too fast.

59. Scenario: Why is "Useful Skew" dangerous in a design with very high On-Chip Variation (OCV)?

- **Answer:** Useful Skew intentionally misaligns the clock to give data more time. However, OCV means clock delays shift randomly due to manufacturing differences. If you intentionally push the clock to the absolute edge to fix timing, a slight OCV shift in the real silicon will instantly push it over the edge and break the chip.

60. Scenario: You have a Setup violation on a path. You swap the driving cell for an Ultra-High Drive cell to push the signal faster. The Setup violation gets *worse*. Why?

- **Answer:** Ultra-High Drive cells are physically massive. They have a huge input capacitance (pin load). By upgrading the cell, you sped up the current net, but you severely overloaded the *previous* cell driving it, creating a worse delay upstream.

Section 7: Advanced Clock Tree Synthesis (CTS)

61. Scenario: Should a Clock Gating Cell (ICG) be placed closer to the main Clock Root, or closer to the Flip-Flops it controls?

- **Answer:** It must be placed closer to the Flip-Flops. The entire point of an ICG is to save power by turning off the clock wire. If you place it near the root, you save power on the whole branch. *However*, the "Enable" signal driving the ICG usually comes from local logic. If the ICG is far away, the Enable signal will arrive late, missing the clock edge and causing a glitch.

62. Scenario: Why do some high-performance chips (like CPUs and Crypto Cores) use a "Clock Mesh" instead of a "Clock Tree"?

- **Answer:** A tree branches out, meaning the end points can have high skew due to OCV. A Clock Mesh literally shorts all the final clock branches together into a giant metal grid across the chip. This guarantees near-zero skew everywhere and extreme stability, but it burns a massive amount of dynamic power.

63. Scenario: You built a clock tree with perfect 0.00ns skew between all flops. However, the Insertion Delay (latency from root to flop) is a massive 8ns. Is this a good clock tree?

- **Answer:** No, it is terrible. A massive insertion delay means the clock passes through dozens of buffers. Because of OCV, every buffer adds a percentage of random delay variation. An 8ns deep tree will have wild, unpredictable skew in actual physical silicon due to temperature and voltage fluctuations.

64. Scenario: Does Global Skew matter more, or does Local Skew matter more?

- **Answer:** Local Skew matters much more. If Flop A and Flop Z are on opposite sides of the chip and never talk to each other, it doesn't matter if their clocks are skewed by 1ns. But if Flop A talks directly to Flop B (local), even a 0.1ns skew can cause a fatal Hold violation.

65. Scenario: What is the difference between Clock Skew and Clock Jitter?

- **Answer: Skew** is spatial (physical). It is the difference in arrival time between two different physical locations on the chip. **Jitter** is temporal (time). It is the fluctuation of the clock period at the *same* location from cycle to cycle, caused by the PLL (Phase Locked Loop) or power supply noise.

66. Scenario: Why do some designers prefer building clock trees entirely out of Clock Inverters instead of Clock Buffers?

- **Answer:** A buffer is just two inverters internally. It is difficult to make a PMOS and NMOS exactly equal, so buffers often have unequal rise and fall times, degrading the clock pulse width. By using single inverters, every rising edge becomes a falling edge at the next stage, naturally canceling out the rise/fall asymmetry.

67. Scenario: You applied "Double Spacing" to your clock nets to prevent crosstalk. At what physical point must you drop the rule and return to normal spacing?

- **Answer:** You must drop the rule right before the leaf level (the pins of the flip-flops). Flip-flop pins are packed tightly at the standard cell level. If you force double-spacing all the way into the pin, the router will fail to connect them because there isn't enough physical room.

68. Scenario: You run CTS. Afterward, your Hold violations explode from 0 to 5,000. Why did CTS cause this?

- **Answer:** Before CTS, the clock arrived at exactly 0.00ns for every flop (ideal clock). CTS builds real wires, introducing Clock Skew. If the clock arrives at the capture flop even a fraction of a nanosecond *before* the launch flop, the data easily races past the clock, causing thousands of hold failures.

69. Scenario: You route a shield wire next to a critical clock net to block crosstalk. Should you connect the shield to VDD (Power) or VSS (Ground)?

- **Answer:** Always VSS (Ground). VSS is the universal quiet reference point. VDD is full of dynamic voltage drops and switching noise. Shielding with VDD would inject power noise directly into your clock line.

70. Scenario: How do you fix a Max Transition violation on a High-Fanout Reset net?

- **Answer:** You cannot just upsize the driver because driving 10,000 flops will overload any cell. You must build a "Reset Tree" (just like a clock tree) by cloning buffers and distributing the load symmetrically.

Section 8: Multi-Voltage, UPF, & Power Intent

71. Scenario: A standard cell in a 0.0V (Shut Down) domain sends a signal to a cell in a 1.0V (Always On) domain. You forgot to add an Isolation Cell. What happens?

- **Answer:** When the first domain shuts down, its output floats (it becomes neither 1 nor 0, but an unpredictable middle voltage). The receiving cell in the 1.0V domain sees this floating input, which partially turns on both its NMOS and PMOS transistors. This creates a direct short to ground (Crowbar Current), burning out the 1.0V domain.

72. Scenario: A signal goes from a 0.75V domain to a 0.95V domain. Where exactly must the Low-to-High Level Shifter be physically placed?

- **Answer:** It must be placed inside the 0.95V domain. A Level Shifter needs access to the higher voltage supply (0.95V) to pull the signal up to the new level. If placed in the 0.75V domain, it wouldn't have access to the required high-voltage rail.

73. Scenario: What is a Retention Flip-Flop?

- **Answer:** When a power domain is shut off to save battery, all flip-flops lose their memory. A Retention Flop contains a tiny "balloon" latch that runs on a separate, always-on backup power supply. Before shut-off, it saves the data and when power returns, it restores the state immediately without needing a full system reboot.

74. Scenario: You placed an Always-On (AON) buffer inside a domain that turns completely off. How does it stay alive?

- **Answer:** AON cells require a secondary, continuous power pin (often called VDD_Backup). The router must physically draw a special power wire through the shutdown domain just to feed this specific cell.

75. Scenario: Domain A and Domain B both operate at exactly 0.95V. However, Domain A is controlled by Power Switch A and Domain B is controlled by Power Switch B. Do you need Level Shifters? Do you need Isolation Cells?

- **Answer:** You do not need Level Shifters because the voltages are identical. However, you absolutely need Isolation Cells. Since they have different switches, Domain A could be turned off while Domain B is awake, creating the floating-input crowbar issue.

76. Scenario: What is the difference between Static IR-Drop and Dynamic IR-Drop?

- **Answer:** Static IR-Drop assumes the chip is sitting idle and calculates voltage drop based on constant leakage current. Dynamic IR-Drop uses a real switching simulation (VCD file) to calculate the massive voltage spikes caused when thousands of flip-flops toggle at the exact same nanosecond.

77. Scenario: You have empty space in the corner of your floorplan. Should you fill it with Decoupling Capacitors (Decaps) to help your power grid?

- **Answer:** No. Decaps act as local batteries that supply instantaneous current during switching. If they are placed far away in an empty corner, the resistance of the wire connecting them to the active logic defeats their purpose. Decaps must be placed as close to the switching hotspots as possible.

78. Scenario: What is "Inrush Current" and how do Power Switches manage it?

- **Answer:** When a sleeping power domain is suddenly turned back on, all the empty wires and cells instantly suck a massive amount of power to charge up. This "inrush" can steal power from the rest of the chip, causing a brownout. Power switches are "daisy-chained" (turned on one by one in a delayed sequence) to slowly charge the domain and prevent grid collapse.

79. Scenario: Why does a slow transition (bad slew rate) cause high dynamic power consumption?

- **Answer:** Every CMOS gate has a threshold where it flips from 0 to 1. During that exact middle moment, both the NMOS and PMOS are slightly open, causing a temporary short-circuit current. If the signal transition is very slow, the gate spends more time in that "middle state," burning excess short-circuit power.

80. Scenario: You are sizing an MTCMOS Power Switch (Header switch). What is the danger of making it too big? What is the danger of making it too small?

- **Answer:** If the switch is too small, its resistance is too high, causing massive IR-drop across the switch when the block is running. If it is too big, the switch itself will have massive leakage power when turned off, defeating the purpose of shutting the block down in the first place.

Section 9: Advanced Routing & Physical Verification

81. Scenario: Your standard cell utilization is only 40% (plenty of empty space), but the router is failing with massive congestion. Why?

- **Answer:** You likely used high pin-density standard cells (complex multi-input logic gates) or you placed macros in a way that formed a narrow "bottleneck" channel. Even with 60% empty space, if all wires are forced through a tiny physical corridor, the metal tracks will overflow.

82. Scenario: An input pin of a logic gate must be permanently tied to logic "1". Why do we use a "Tie-High Cell" instead of just wiring the pin directly to the VDD power rail?

- **Answer:** Connecting the thin gate oxide of a transistor directly to the thick, noisy VDD rail makes it highly vulnerable to ESD (Electrostatic Discharge) and voltage spikes, which will blow the gate. A Tie-High cell contains a resistor-like internal structure to safely deliver the logic "1" without the electrical risk.

83. Scenario: What is the exact difference between a Placement "Fence" and a Placement "Region"?

- **Answer:** A **Fence** is an exclusive hard constraint: specified cells MUST go inside and no other cells are allowed in. A **Region** is inclusive: specified cells MUST go inside, but if there is leftover room, the tool is allowed to place other random cells in there too.

84. Scenario: Net A and Net B are routed side-by-side. Net A is totally quiet (static 0). Net B suddenly switches from 0 to 1. Because of crosstalk capacitance, Net A experiences a "Glitch" and spikes up to 0.4V. Will the chip fail?

- **Answer:** It only fails if 0.4V crosses the threshold voltage of the receiving gate. If the receiver interprets 0.4V as a logic "1", it will register a false data point. If the receiving cell has a high threshold (e.g., 0.5V), it will ignore the 0.4V glitch and the chip will survive.

85. Scenario: What is the difference between Crosstalk Glitch and Crosstalk Delay?

- **Answer: Glitch** happens when the victim net is sitting perfectly still and the aggressor causes it to bounce. **Delay** happens when *both* nets are switching at

the exact same time. If they switch in opposite directions, they fight each other and slow the signal down.

86. Scenario: What is the difference between a Placement HALO and a Routing Blockage?

- **Answer:** A HALO is drawn around a macro; it stops the tool from placing any standard cells too close to the macro's edge. A Routing Blockage stops the tool from drawing metal wires across that specific area.

87. Scenario: The router inserts an Antenna Diode to fix an antenna violation on a long wire. Should the diode be placed near the driving cell, or near the receiving cell?

- **Answer:** Near the receiving cell. The danger of the antenna effect is the static charge blowing out the delicate gate oxide of the *receiving* transistor. The diode must be as close to that gate as possible to siphon off the charge.

88. Scenario: What exactly happens during the "Track Assignment" phase of routing?

- **Answer:** Global Routing divides the chip into grids and plans the rough path. Track Assignment is the middle step: it looks at the rough path and assigns the wire to a specific, continuous physical metal track (e.g., Metal 4, Track 12) to minimize vias, before Detailed Routing finishes the exact pin connections.

89. Scenario: Your wires are perfectly connected using single vias. The tool goes back and spends hours swapping them to "Redundant Vias" (double vias). Why waste the space?

- **Answer:** Vias are microscopic holes filled with metal. During manufacturing, a single via can easily fail to fill properly, breaking the net entirely. Dropping a second via right next to it provides a backup path, vastly improving the manufacturing yield of the chip.

90. Scenario: At advanced nodes (7nm/5nm), standard cells are so incredibly small that the router cannot physically drop vias to connect to their pins. How is this solved?

- **Answer:** The foundry provides a predefined "Pin Access" pattern. The placement engine must align the cells strictly on a specific grid so that the pins perfectly match up with the predefined safe zones where the router is legally allowed to drop a microscopic via.

Section 10: Signoff, Parasitics & Verification

91. Scenario: Your design perfectly passed Setup and Hold timing in PrimeTime. However, when the physical chip is manufactured, it fails Setup timing on the test bench. What did your signoff miss?

- **Answer:** You likely missed **Dynamic Voltage Drop (IR Drop)**. If your PrimeTime analysis used ideal voltage (e.g., 0.95V), but the actual chip sags to 0.85V under heavy load, the transistors will slow down significantly in the real world, causing Setup failures that were invisible to STA.

92. Scenario: You run LVS (Layout vs. Schematic) and get a "Soft Error". What does this mean?

- **Answer:** A Soft Error usually means there is an electrical connection happening through the silicon substrate or N-well, rather than through a hard metal wire. The tool flags it because substrate connections have incredibly high resistance and are usually mistakes (like a missing tap cell).

93. Scenario: DRC flags a "Minimum Metal Area" violation. The wire is wide enough and spaced correctly, but it is just a tiny little square of metal. Why does the foundry care?

- **Answer:** During CMP (Chemical Mechanical Polishing), the wafer is scrubbed flat. If a piece of metal is too small in total surface area, the scrubbing pad will literally peel the tiny metal flake right off the chip, breaking the connection.

94. Scenario: Electromigration (EM) rules are incredibly strict for Power Nets, but much more relaxed for Signal Nets. Why?

- **Answer:** Power nets experience **DC Current** (electricity flows in one constant direction), which constantly pushes the metal atoms like a river, eventually breaking the wire. Signal nets experience **AC Current** (toggling 1 and 0), meaning the current reverses direction constantly, pushing the atoms back and forth, which causes far less permanent damage.

95. Scenario: During SPEF extraction, you extract parasitic RC values at Cbest (Best Capacitance) and RCworst (Worst RC). Which corner should you use to check for Setup timing on a very long data wire?

- **Answer:** You use RCworst. A long wire is dominated by resistance (R). RCworst maximizes the resistance, resulting in the slowest possible signal travel time, which is the absolute worst-case scenario for Setup.

96. Scenario: What is the difference between Base DRC and Metal DRC?

- **Answer:** Base DRC checks the rules for the silicon layers at the bottom of the chip (N-wells, P-substrate, Poly-silicon gates, FinFETs). Metal DRC checks the rules for the copper routing layers built on top of the silicon.

97. Scenario: Before final tapeout, the tool drops thousands of random, useless metal rectangles all over your empty routing channels. What is this and why do we do it?

- **Answer:** This is **Dummy Metal Fill**. To polish the wafer flat during manufacturing, the metal density must be uniform across the entire chip. If a region has no metal, the polishing pad will "dish" or scoop out the silicon. Dummy fill ensures the pad stays level.

98. Scenario: Can IR-Drop (low voltage) ever cause a Hold violation?

- **Answer:** Yes, but in a specific way. Usually, low voltage slows data down, which actually *helps* Hold time. However, if the IR-drop occurs on the **Clock Tree**, the clock buffers will slow down. If the capture clock is delayed by IR-drop, the data will race past it, creating a fatal Hold violation.

99. Scenario: You run metal fill and now PrimeTime reports new Setup violations. How did adding unconnected dummy metal ruin your timing?

- **Answer:** Dummy metal is physical copper. When placed next to a critical timing net, it increases the side-wall capacitance of that net (coupling capacitance). That extra capacitance slows the signal down, pushing your Setup margin over the edge.

100. Scenario: You are stuck in the "Signoff ECO Loop": You fix a Hold violation by adding a buffer, but that slows the path down and causes a Setup violation. You remove the buffer to fix Setup and Hold fails again. How do you break this loop?

- **Answer:** You must step away from buffering. Options include:
    1. **LVT Swapping:** Swap the data cells to Low-VT (faster). This fixes Setup, giving you enough margin to safely insert the Hold buffer.
    2. **Downsizing:** Instead of adding a buffer to fix Hold, slightly downsize the driving cell. This naturally slows the data down just a fraction, fixing Hold without adding the massive delay of a whole new gate.
    3. **Useful Skew:** Push the capture clock later to fix Setup, or pull it earlier to fix Hold, avoiding touching the data path entirely.